Numerical Analysis (089180)
Prof. Carlo de Falco
carlo.defalco@polimi.it

A.Y. 2024-2025
Giuseppe Caliò
giuseppe.calio@polimi.it

Lab 6 – Solutions

October 28, 2024

This lab deals with the numerical solution of a linear system

$$\mathbb{A}\boldsymbol{x} = \boldsymbol{b}, \quad \text{for } \mathbb{A} \in \mathbb{R}^{n\times n}, \boldsymbol{b} \in \mathbb{R}^n$$

We will focus on stationary iterative methods.

1 Fill-in of a matrix

Definition
"In general, the LU factorization process does not preserve the sparsity pattern of the original matrix A, as it may generate non-null elements in L and U in some positions (i, j) where the corresponding original elements aij were null. This phenomenon is called fill-in and depends on both the original pattern of A and the values of its entries" (A. Quarteroni et al., Scientific Computing with MATLAB and Octave)

Exercise 6.1 (Exam July 20, 2023). Consider the following square matrix $\mathbb{A} \in \mathbb{R}^{n\times n}$:

$$\mathbb{A} = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 & 1 \\ 0 & 1 & 0 & \dots & 0 & 1 \\ & & \ddots & & 0 & \vdots \\ 0 & & \dots & 1 & 0 & 1 \\ 0 & & \dots & 0 & 1 & 1 \\ 1 & 2 & \dots & 5 & 6 & 7 \end{bmatrix}.$$

Exam July 20, 2023

1. Using the Matlab commands `diag` and `ones`, construct the matrix $\mathbb{A}$. Provide the Matlab commands used.
2. Using the Matlab command `lu`, calculate the LU factorization of the previously constructed matrix. Provide the obtained factors $\mathbb{L}$ and $\mathbb{U}$, along with the Matlab commands used.
3. Using the Matlab command `spy`, answer the following questions, providing reasoning: Was pivoting performed? Did fill-in occur? Provide the Matlab commands used.
4. Given the known term $\boldsymbol{b} = [1, 0 \dots 0]^T \in \mathbb{R}^n$, solve the linear system $\mathbb{A}\boldsymbol{x} = \boldsymbol{b}$ using the functions `forwardsubstitution.m` and `backwardsubstitution.m`. Provide the obtained solution and the Matlab commands used.
5. Replace the last row of matrix $\mathbb{A}$ with the vector $\boldsymbol{v} = [1.1, 1 \dots 1] \in \mathbb{R}^n$ and repeat steps a, b and d for n=20,30,40. Knowing that the exact solution is $\boldsymbol{x} = [1 - \frac{1.1}{1.1+n-3}, -\frac{1.1}{1.1+n-3}, \cdots - \frac{1.1}{1.1+n-3}, -\frac{1.1}{1.1+n-3}]^T \in \mathbb{R}^n$, calculate the relative error of the solution for each n.(Hint: use the Matlab command `ones` to create the components frome the second to the second-to-last element). Using the Matlab command `lu`, calculate the LU factorization of the new matrix. Provide the obtained error, along with the Matlab commands used.

Solution Exercise 6.1.

ex_6_1.m

```
clc;clear;close all;
%% Point a: build A
n=7;
A=diag(ones(n, 1));
A(1:end, end)=1;
A(end, :)=1:n;
%%%%%%%%%%%%%%%%%%
%Check
fprintf("A is:\n")
disp(A)

%% Point b: LU factorization
[L, U, P]=lu(A);

fprintf("L is:\n")
disp(L)
fprintf("U is:\n")
disp(U)

%% Point c
figure("Name",sprintf("Pattern of Matrix P"))
spy(P)
% We observe that pivoting has been performed; in particular,
% the last row of A has become the second row,
% and the rows from 2 to 6 of A have become rows 3 to 7.
% See also Figure 1.

figure("Name",sprintf("Pattern of Matrix L"))
spy(L)
figure("Name",sprintf("Pattern of Matrix U"))
spy(U)
% We also observe the occurrence of the fill-in phenomenon,
% as the matrix U is full despite the sparsity
% of matrix A. See Figure 2 and 3.

%% Point d
b=[1; zeros(n-1, 1)];


y=forward_substitution(L, P*b);
x=backward_substitution(U, y);

fprintf("The solution is x= \n");
disp(x)
% Check
% disp(A\b)

%% Point e
err=[];
a=1.1;
for n=[20 30 40]
   A=diag(ones(n, 1));
   A(1:end, end)=1;
   v = [1.1; ones(n-1,1)];
   A(end, :)=v;
   tmp=a/(a+n-3);
   xex=[1-tmp ; -repmat(tmp, n-2,1); tmp];
   %xex=[1-tmp; -tmp*ones(n-2,1); tmp];
   [L, U, P]=lu(A);

   b=[1; zeros(n-1, 1)];

   y=forward_substitution(L, P*b);
   x_comp=backward_substitution(U, y);
   err_n=norm(xex-x_comp)/norm(xex);
   err=[err err_n];

end
err
```

2 Richardson Methods

Richardson methods consist in computing the sequence

$$\boldsymbol{x}^{(k)} = \boldsymbol{x}^{(k-1)} + \alpha\, \mathbb{P}^{-1}\, \boldsymbol{r}^{(k-1)}$$

where $\mathbb{P}$ is a suitable non singular **preconditioning matrix**, $\boldsymbol{r}^{(k-1)} = \boldsymbol{b} - \mathbb{A}\boldsymbol{x}^{(k-1)}$ is the **residual** vector associated with the $(k-1)$-th iteration and $\alpha > 0$ is the **acceleration parameter** of the method.

Richardson schemes are a generalization of the methods already presented,

$$\boldsymbol{x}^{(k+1)} = \mathbb{B}\boldsymbol{x}^{(k)} + \boldsymbol{g},$$

where the iteration matrix is

$$\mathbb{B}(\alpha) = \mathbb{I} - \alpha\mathbb{P}^{-1}\mathbb{A}$$

Convergence properties depend on the coefficient matrix $\mathbb{A}$ and method parameters α and $\mathbb{P}$.

- $\alpha = 1$, $\mathbb{P} = \mathbb{D}$: Jacobi ($\mathbb{B}_J = \mathbb{I} - (\mathbb{D})^{-1}\mathbb{A}$)
- $\alpha = 1$, with $\mathbb{P} = \mathbb{D} - \mathbb{E}$: Gauss-Seidel ($\mathbb{B}_{GS} = \mathbb{I} - (\mathbb{D} - \mathbb{E})^{-1}\mathbb{A}$)

Theorem 6.1. *For any non singular preconditioning matrix $\mathbb{P}$, the stationary Richardson method is convergent for any initial guess $\boldsymbol{x}^{(0)}$ iff*

$$\frac{2Re(\lambda_i)}{\alpha|\lambda_i|^2} > 1, \quad \forall i = 1.\ldots, n,$$

where $\lambda_i \in \mathbb{C}$ are the eigenvalues of $\mathbb{P}^{-1}\mathbb{A}$.

The result of this theorem comes from asking the spectral radius of the iteration matrix being less than one.

3 Choice of the parameter α: optimality

Theorem 6.2. *Let $\mathbb{A}$ and $\mathbb{P}$ be symmetric positive definite matrices. Then stationary Richardson methods are convergent for any initial guess if and only if the parameter α is selected in the $[0, 2/\lambda_{\max}]$ interval, being $\lambda_{\max}$ the maximum eigenvalue of $\mathbb{P}^{-1}\mathbb{A}$. Moreover the spectral radius of the iteration matrix is minimized for $\alpha = \alpha_{opt}$, where*

$$\alpha_{opt} = \frac{2}{\lambda_{\min} + \lambda_{\max}}.$$

and $\lambda_{\min}$ the minimum eigenvalue of $\mathbb{P}^{-1}\mathbb{A}$. The spectral radius of the iteration matrix $\mathbb{B}(\alpha)$ is minimum if $\alpha = \alpha_{opt}$, with

$$\rho_{opt} = \frac{\lambda_{max} - \lambda_{min}}{\lambda_{min} + \lambda_{max}}.$$

Exercise 6.2. Consider the linear system $\mathbb{A}\boldsymbol{x} = \boldsymbol{b}$, with $\boldsymbol{b} = 0.2*$`ones(50,1)` and

$$\mathbb{A} = \begin{bmatrix} 4 & -1 & -1 & & & \\ -1 & 4 & -1 & -1 & & \\ -1 & -1 & 4 & -1 & -1 & \\ & \ddots & \ddots & \ddots & \ddots & \\ & & -1 & -1 & 4 & -1 \\ & & & -1 & -1 & 4 \end{bmatrix}, \quad \mathbb{T} = \begin{bmatrix} 2 & -1 & & & & \\ -1 & 2 & -1 & & & \\ & -1 & 2 & -1 & & \\ & \ddots & \ddots & \ddots & \ddots & \\ & & & -1 & 2 & -1 \\ & & & & -1 & 2 \end{bmatrix}.$$

a. Are $\mathbb{A}$ and $\mathbb{T}$ symmetric positive definite matrices?

b. Apply the Richardson method starting from $\boldsymbol{x}^{(0)} = \boldsymbol{0}$, with a tolerance equal to 10^{-6} and $\mathbb{P} = \mathbb{I}$. Compare the required number of iterations (if the method is convergent) for $\alpha = 0.2$, $\alpha = 0.33$ and $\alpha = \alpha_{opt}$.

c. Apply the Richardson method starting from $\boldsymbol{x}^{(0)} = \boldsymbol{0}$, with a tolerance equal to 10^{-6}, $\mathbb{P} = \mathbb{T}$, $\alpha = \alpha_{opt}$. Compare the required number of iterations with the one associated with the non-preconditioned case. Compare the condition numbers of $\mathbb{A}$ and $\mathbb{P}^{-1}\mathbb{A}$.

Solution Exercise 6.2.

ex_6_2.m

```
%% Exercise 6.1
clc;clear all;close all;
%% Point a
n = 50;
A = 4*eye(n) - diag(ones(n-1, 1), 1) - diag(ones(n-1, 1), -1) - diag(ones(n-2, 1), 2) - diag(
ones(n-2, 1), -2);
b = 0.2*ones(n, 1);
I = eye(n);
T = 2*eye(n) - diag(ones(n-1, 1), 1) - diag(ones(n-1, 1), -1);

% Both matrices are clearly symmetric
eig_A = eig(A);
eig_T = eig(T);

if all(eig_A>0) && all(eig_T>0)
% The fact that all eigenvalues are positive reveals that they are also positive definite.
   fprintf("Positive definite Matrices\n");
end

%% Point b
% Apply the Richardson method starting from x = 0, with a tolerance equal to
% 10 6  and P = I. Compare the required number of iterations (if the method is
% convergent) for alpha = 0.2, alpha = 0.33 and alpha = alpha_opt.

x0 = zeros(n, 1); tol = 1e-6; maxit = 10000;
% b.1
alpha = 0.2;
rho_1=max(abs(eig(I - alpha*A))) % less than 1--> convergence OK
% or since the Matrix A is s.d.p.
% alpha_max= 2/max(eig(P\A)); %alpha<alpha_max---> convergent
[x1, iter1, incr1] = prec_rich_method(A, b, I, alpha, x0, tol, maxit)

%b.2
alpha = 0.33;
rho_2=max(abs(eig(I - alpha*A))) % greater than 1 --> NOT convergent
% You can check that this alpha is s.t. alpha>2/max(eig_A)
%[x2, iter2, incr2] = prec_rich_method(A, b, I, alpha, x0, tol, maxit) % we already know that
it will not converge

alpha = 2/(min(eig_A) + max(eig_A));
rho_3=max(abs(eig(I - alpha*A))) % less than 1-->optimal convergence
[x3, iter3, incr3] = prec_rich_method(A, b, I, alpha, x0, tol, maxit)

% Among the proposed values, the minimum number of iterations is obtained
% at alpha = alpha_opt

%% Apply the Richardson method starting from x = 0, with a tolerance equal to
% 10 6  , P = T,  = opt . Compare the required number of iterations with the one
% associated with the non-preconditioned case. Compare the condition numbers of A
% and P\A.
x0 = zeros(n, 1); tol = 1e-6; maxit = 10000;

Tinv_A = T\A;
eig_Tinv_A = eig(Tinv_A);
alpha = 2/(min(eig_Tinv_A) + max(eig_Tinv_A));
rho_4=max(abs(eig(I - alpha*Tinv_A))) % less than 1
[x4, iter4, incr4] = prec_rich_method(A, b, T, alpha, x0, tol, maxit)

cond_A = cond(A)
cond_Tinv_A = cond(Tinv_A)

% Preconditioning the linear system with T results both in a
% better performance of the method (less iterations are needed)
% and a smaller condition number.
```

Exercise 6.3. Consider

$$\mathbf{A} = \begin{bmatrix} 2 & 1 & 1 & 1 & \dots & 1 \\ 4 & 4 & 1 & 1 & \dots & 1 \\ 8 & 8 & 8 & 1 & \dots & 1 \\ \vdots & \vdots & \vdots & \ddots & \ddots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \ddots & 1 \\ 2^n & \dots & \dots & \dots & \dots & 2^n \end{bmatrix} \qquad \mathbf{b} = \begin{bmatrix} 1 \\ 2 \\ \vdots \\ n \end{bmatrix} \qquad \text{con } n = 20.$$

Solve the linear system $\mathbf{Ax} = \mathbf{b}$ by using three different preconditioned stationary Richardson schemes: $\mathbf{P}_1$: (no preconditioner), $\mathbf{P}_2$: (Jacobi), $\mathbf{P}_3$: (Gauss-Seidel). Set `toll=1e-6`, `nmax=1e3`, $\mathbf{x}_0 = [1, 1, \dots, 1]^T$. Select the acceleration parameter α as the midpoint of the convergence interval. *Hint*: in case of **complex eigenvalues** of the preconditioned matrix, if their real part is always positive, then the method converges for $0 < \alpha < 2 \min \frac{Re(\lambda_i)}{|\lambda_i|^2}$.

Solution Exercise 6.3.

ex_6_3.m

```
%% Exercise 6.2
clear; close all;clc;
%% Build the matrix
n = 20;
A = ones(n);
for i = 1:n
   A(i,1:i) = 2^i;
end
b = (1:n)';
x_ex = A\b;
%% Solve the system with tree different preconditioners
% convergence iff 0 < alfa < 2min(Re(eig)/abs(eig)^2)
%% P=I
P=eye(n); x0=ones(n, 1); toll=1e-6; nmax=1e3;
eigs=eig(P\A);
alpha_max=2*min(real(eigs)./abs(eigs).^2);
alpha1=alpha_max/2;
[x1, it1, res1] = richprec(A,b,P, alpha1, x0,nmax, toll);
fprintf("Iterations: %d\t", it1)
fprintf("Normalized Residual: %d\t", res1(end))
fprintf("Condition Number: %d\n", cond(P\A))

%% P=D
P=diag(diag(A));
eigs=eig(P\A);
alpha_max=2*min(real(eigs)./abs(eigs).^2);
alpha2=alpha_max/2;
[x2, it2, res2] = richprec(A,b,P, alpha2, x0,nmax, toll);
fprintf("Iterations: %d\t", it2)
fprintf("Normalized Residual: %d\t", res2(end))
fprintf("Condition Number: %d\n", cond(P\A))

%% P=D-E
P=tril(A);
eigs=eig(P\A);
alpha_max=2*min(real(eigs)./abs(eigs).^2);
alpha3=alpha_max/2;
[x3, it3, res3] = richprec(A,b,P, alpha3, x0,nmax, toll);
fprintf("Iterations: %d\t", it3)
fprintf("Normalized Residual: %d\t", res3(end))
fprintf("Condition Number: %d\n", cond(P\A))
```

Exercise 6.4 (Exam July 11, 2024). Consider the linear system $\mathbb{A}\boldsymbol{x} = \boldsymbol{b}$, with $\mathbb{A} \in \mathbb{R}^{20\times 20}$, symmetric positive definite, and $\boldsymbol{b} \in \mathbb{R}^{20}$ given by $\mathbb{A} =$`pentadiag(-1,-4,10,-4,-1)` and $\boldsymbol{b} = [1, 1 \dots 1]^T$. Here, `pentadiag` refers to a pentadiagonal matrix, meaning it has only the 5 central diagonals different from zero, filled with the given values.

Exam July 11, 2024

1. Solve the proposed system using the stationary Richardson method with $\mathbb{P} = \mathbb{I}$(use the function `richardson.m`), $\alpha = 0.25$ initial guess $\boldsymbol{x}^{(0)} = \boldsymbol{b}$, tolerance $tol = 10^{-3}$ and $n_{max} = 500$. Report the number of iterations N performed, the third component $\boldsymbol{x}_3^{(N)}$ of the approximated solution vector $\boldsymbol{x}^{(N)}$ and the value of the corresponding relative residual $\boldsymbol{r}_3^{(N)}$. Use format long. Also, provide the Matlab instructions used.
2. Comment on the results obtained in the previous point, providing appropriate theoretical explanations on how such considerations could have been anticipated a priori.
3. Rerun the method as in point a, but now with the optimal value of the parameter α_{opt}. Report this value, the number of iterations required by the method, a comment on the results, and the Matlab instructions used.
4. Now solve the problem from point a using $\alpha = 0.84$ with the preconditioned Richardson method with $\mathbb{P} = tridiag(-5, 10, -5)$ Here, tridiag refers to a tridiagonal matrix, meaning it has only the 3 central diagonals different from zero, filled with the given values. Provide the Matlab instructions, the number of iterations used, and a comment on the results.

Solution Exercise 6.4.

ex_6_4.m

```
%% Exercise 6.4
clc;clear; close all;
format long
%% Build the matrix
n=20;
A = -diag(ones(n-2,1), -2) - 4* diag(ones(n-1,1), -1) + 10*diag(ones(n, 1), 0)...
   -diag(ones(n-2,1), +2) - 4* diag(ones(n-1,1), +1);
%disp(A)
b=ones(n,1);
%% Solve Using the non preconditioned Richardson Method
alpha=0.25; x0=b; tol=1e-3; nmax=500;

[x, iter, res] = richardson(A, b,x0, alpha, tol, nmax);
fprintf("Iterations: %d\n",iter);
fprintf("Third component of x: %d\n", x(3))
res_xn=(b-A*x)/norm(b);
fprintf("Third component of the residual of x: %d\n", res_xn(3))

%% Point b
% The residual is:
%res=norm(b-A*x)/norm(b);
fprintf("Normalized Residual of x in 2-norm: %d\n", res(end))
% As can be seen from this result and the number of iterations taken by the method
% (500, which is the maximum number of iterations),
% we deduce that the Richardson method did not converge
% within the maximum number of iterations
% and that the solution is diverging.
% This could have been anticipated beforehand,
% as the relaxation parameter used,alpha=0.25
% is greater than the maximum allowed value
eigs=eig(A);
% alpha_max= 2*min((real(eigs)./(abs(eigs).^2)));
% Since the eigenvalues are real (you can check with isreal(eigs))
alpha_max=2/max(eigs);
fprintf("Maximum Alpha: %d\n",alpha_max)

%% Point c
alpha_opt=2/(max(eigs)+min(eigs));
fprintf("Optimal Alpha: %d\n",alpha_opt)
[x2, iter2, incr2] = richardson(A, b,x0, alpha_opt,tol, nmax);
fprintf("Iterations: %d\n",iter2);

% In this case, the solution converges within the maximum number of iterations.
% The convergence here is ensured by using a value of alpha such that
% 0<alpha<2/lamda_max
% In particular, alpha_opt guarantees the least number of iterations.

%% Point d
alpha2= 0.84;
P=-5*diag( ones(n-1,1), -1) + 10*diag( ones(n,1), 0) - 5*diag( ones(n-1,1), +1);
```

```
[x3, iter3, incr3] = richprec(A, b, P, alpha2, x0,nmax, tol);
fprintf("Iterations: %d\n",iter3);
% In this case, the Richardson method is very fast.
% From theory, we know that if the preconditioner is a good approximation of A,
% then the number of iterations decreases.
% This is the case here, as P has the same diagonal as A,
% and the diagonals just above and below it contain the sums of the other diagonals of A.
% Furthermore, from theory, we know that if the linear system associated
% with P (for calculating the residual) is easy to solve,
% then the total cost is significantly reduced.
% This should be our case since P has a simple structure (tridiagonal).
```

Exercise 6.5. Consider the following matlab commands:

$$\begin{cases} \texttt{n=20; A=eye(n);} \\ \texttt{A(1,:)=A(1,:)-1/n/10;} \\ \texttt{A(:,1)=A(:,1)-1/n/10;} \\ \texttt{b=b(2:n,1) = (n*10 + 1)/(n*10);} \\ \texttt{b(1) = (11*n + 1)/(n*10);} \\ \texttt{xex = ones(n,1);} \end{cases}$$

1. compare the sparsity pattern of the exact (LU) and inexact (ILU) factorization;
2. Check the convergence of the stationary Richardson method, adopting P=I and P equal to the incomplete factorization of A and then solve the system; Set $\alpha = 1.9$, $x^{(0)} = 0$, $tol = 1e-3$, $nmax = 200$. Compare the number of iterations and compute the relative error of the solution;
3. Repeat the previous point with the optimal value of alpha;

Solution Exercise 6.5.

ex_6_5.m

```
%% Exercise 6.5
clc;clear all ;close all
%% Point a
n = 20;
A = eye(n);
A(1, :) = A(1, :) + 1/n/10;
A(:, 1) = A(:, 1) + 1/n/10;

figure(1)
spy (A)


b(2:n,1) = (n*10 + 1)/(n*10);
b(1) = (11*n + 1)/(n*10);

xex = ones (n, 1);


%% compare exact and inexact factorization
[L, U] = lu (A);
[Li, Ui] = ilu (sparse(A));

figure('Name', 'LU Factorization')
subplot(1,2,1)
spy (L)
title('L')
subplot(1,2,2)
spy (U)
title('U')

figure('Name', 'Incomplete LU Factorization')
subplot(1,2,1)
spy (Li)
title('incomplete L')
subplot(1,2,2)
spy (Ui)
```

```
title('incomplete U')

%% SOlve the system with P=I and P=LiUi;
eigs_p=eig(((Ui*Li)\A));
alpha_max=2/(max(eigs_p));
eigs=eig(A);
alpha_max=2/(max(eigs));
alpha = 1.9;

x0 = zeros(size (b)); tol=1e-3; max_it=200;
%non preconditioned
[x1, iter1, incr1] = prec_rich_method(A, b, eye(n), alpha, x0, tol, max_it);
res1=(b-A*x1);
resnorm1 = norm (res1, 2);


% preconditioned with ILU
[x2, iter2, incr2] = prec_rich_method(A, b, Li*Ui, alpha, x0, tol, max_it);

res2=(b-A*x2);
resnorm2 = norm (res2, 2);


%% Again with the optimal value of alpha


alpha_opt_p=2/(max(eigs_p)+min(eigs_p));
alpha_opt=2/(max(eigs)+min(eigs));

[x3, iter3, incr3] = prec_rich_method(A, b, eye(n), alpha_opt, x0, tol, max_it);
[x4, iter4, incr4] = prec_rich_method(A, b, Li*Ui, alpha_opt_p, x0, tol, max_it);
```